Musical note classification using MediaPipe

Curso 2k25/2k26

Apellidos, nombre	Michał P
Titulación	Grado de Ingeniería informática (Erasmus - I am not assigned to a specific module)
Fecha	Abril 2026

Index

1 Core concepts description..... 1
2 Introduction..... 1
3 Objectives..... 1
4 Time schedule..... 2
5 Development..... 2
5.1 Obtaining frames from the webcam and displaying image using OpenCV..... 2
5.2 Hand recognition with MediaPipe..... 4
5.3 Classifying the notes with scikit-learn..... 7
5.4 Gluing it together..... 8
6 Possible future expansions..... 12
7 Conclusion..... 12
8 Bibliography and references..... 12

1 Core concepts description

This article focuses on a system that enables the user to interact with a computer system using OpenCV [6,7] as the capture and display system and MediaPipe [1,2,3,4] as the library that uses machine learning to detect hand gestures. Another library - scikit-learn [5] is used to recognize gestures made by the user.

The program or rather the group of programs is an interactive application that uses computer vision to first learn how the user plays a given note on a recorder to later classify them using machine learning. We won't go into great depths explaining openCV, scikit-learn or MediaPipe, but their usage together will nevertheless be interesting. The program is running on the Linux platform with python 3.14, OpenCV 4.13, MediaPipe 0.10.33, scikit-learn 1.7.2. However it can be run on Windows and MacOS without much trouble as well as other platforms supported by python as long as the device also has some sort of a video camera.

To get the full understanding of the application looking at source code is needed. The code fragments provided here are often just fragments of the whole program because the code would occupy too much space. Some fragments are also simplified to make the reader direct their focus at a specific feature of the whole program.

2 Introduction

The article will show how human computer interaction works in the case of gesture recognition and how it can be used to teach how to play notes on a flute and perhaps other musical instruments. The computer takes the input through a video camera, runs the input through MediaPipe and depending on which mode is chosen either trains a neural network or classifies which note is being played.

We'll explain openCV video capture parts, then MediaPipe gesture processing and then last but not least the scikit-learn machine learning parts. All the parts linked together will be shown working together creating a functioning note classifying program.

The program was created for Sistemas Multimedia Interactivos e Inmersivos subject. The idea for it was taken from the list of topics proposed by the professor.

3 Objectives

Once read the reader will be able to:

- Explain the flow of the program
- Explain the parts played by different components of the program

- Propose extra functionalities that could be added in the future
- Explain the limitations of the program and their sources

4 Time schedule

Originally I wanted to try detecting the holes in the flute, but it turns out holes are only visible when there is no finger covering them. Since this project was supposed to use MediaPipe I couldn't rely on counting the number of holes and the distance between them to classify notes. I tried just feeding a machine learning algorithm with positions detected by MediaPipe and it worked quite well so this ended up becoming the final solution. This all was done during the Easter break as indicated in the schedule.

After that I consulted the professor about the things he would like to see added and so on. The remaining work was completed during the weekend before the final presentation. The functionality requested by the professor was to add a "game" mode that would ask the user to position their fingers in a position to play a displayed note and to add a buffer which would prevent notes from being rapidly switched.

5 Development

This section describes the libraries used one by one and then shows how they all work together to create the final solution.

5.1 Obtaining frames from the webcam and displaying image using OpenCV

It doesn't matter which part of the program we take a look at - all of them require the camera as the input source for this purpose OpenCv was used. It's also used for managing the source frame (flipping it etc.) and displaying the text and images and finally managing the window and its input.

1. Opening the camera
2. Getting a frame
3. converting the format from BRG (used by OpenCV) to RGB (used by MediaPipe)
4. converting the format back on the frame gotten back from MediaPipe
5. flipping the frame
6. adding text and images
7. displaying the frame in a windows
8. handling the input (the window may be closed by pressing q)
9. freeing the resources once the user decided to terminate the program

Listing 1: Typical usage of OpenCV in our program

The order of these operations is crucial. Firstly we absolutely shouldn't add the ui before passing the image to MediaPipe. This may cause the hands to be obstructed which is not a problem for the user, but it makes detecting them much harder for MediaPipe. We also mustn't forget about converting the image to a proper colorspace since openCV uses BRG due to historical reasons, while MediaPipe uses today's RGB standard. FInally We have to be really careful when flipping the image, it must be done before drawing the ui but after passing the image to MediaPipe - MediaPipe [3] expects the image to be mirrored if we flipped the image before MediaPipe processing our left and right hand would be switched - this may cause bugs if we decided to process the hands differently (as we do in our program). A question may arise - why flip the image in the first place if it may cause so many problems? The answer is quite simple - the users are used to seeing themselves in the mirror and expect the webcams to work in the same way. It's quite unusual to see oneself in any other perspective, it may cause confusion and discomfort.

```
import cv2

def cleanup(cap):
    if cap is not None: # handle cleanup
        cap.release()
    cv2.destroyAllWindows()

ref = cv2.imread("notes.png") # load image
ref = cv2.resize(ref, (200, int(200 * ref.shape[0]/ref.shape[1])))

cap = cv2.VideoCapture(0) # open camera

while True:
    ret, frame = cap.read() # get frame

    if not ret:
        cleanup(cap)

    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # change the
colorspace for MediaPipe
    # here is the place to put MediaPipe functions and other auxiliary
libraries
    frame = cv2.cvtColor(rgb_frame, cv2.COLOR_RGB2BGR) # change the
colorspace back to the one used by OpenCV

    frame = cv2.flip(frame, 1) # flip frame
    cv2.putText(frame, "example message", (20, 50), 2, 1, (0, 255, 0), 2)
```

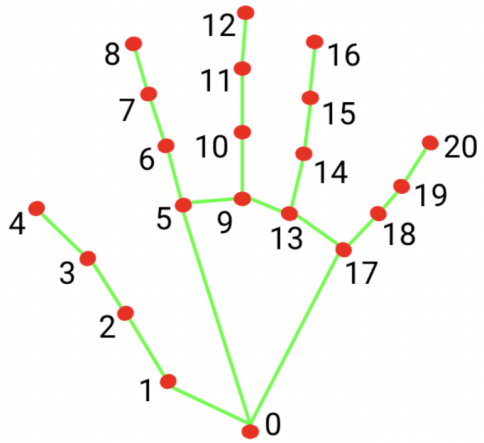
```
# display text
   frame[10:10+ref.shape[0], -210:-10] = ref # display image
   cv2.imshow('example window', frame) # display the frame in a windows

   if cv2.waitKey(1) & 0xFF == ord('q'): # handle input
       cleanup(cap)
```

Listing 2: This is what the typical flow of openCV looks like in described programs. All the important steps are described in the comments

5.2 Hand recognition with MediaPipe

Hand recognition is the key process which the whole program relies on. Using MediaPipe we can track whole bodies, hands and even faces with high levels of accuracy using nothing but an ordinary webcam. However the hands part is the only one that interests us in this case. MediaPipe [2] hand recognition works by using a so-called "landsmarks system" - landmark. A landmark is just a point with coordinates, which is supposed to represent a specific point of our hand (usually a joint, but not always). The coordinates indicate its - and thus this part's of our hand position in 3d space. Mediapipe also recognizes which hand is being detected and supports tracking both of them at the same time.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

Figure 2: landmarks assigned to a hand. Both hands use the same set of landmarks.

The detection is done by loading a model with the chosen option and the calling detect() function on a frame.

```
import cv2
import mediapipe as mp
from mediapipe.tasks.python import vision
from mediapipe.tasks import python
from shared import draw_landmarks_on_image
```

```
options = vision.HandLandmarkerOptions(
base_options=python.BaseOptions(model_asset_path="hand_landmarker.task")
,   running_mode=vision.RunningMode.IMAGE, num_hands=2)

def cleanup(cap):
   if cap is not None: # handle cleanup
       cap.release()
   cv2.destroyAllWindows()

ref = cv2.imread("notes.png") # load image
ref = cv2.resize(ref, (200, int(200 * ref.shape[0]/ref.shape[1])))

cap = cv2.VideoCapture(0) # open camera

with vision.HandLandmarker.create_from_options(options) as detector:
   while True:
       ret, frame = cap.read()

       if not ret:
           cleanup(cap)

       rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
       res = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB,
data=rgb_frame))
       annotated = draw_landmarks_on_image(rgb_frame, res)
       frame = cv2.cvtColor(annotated, cv2.COLOR_RGB2BGR)
       frame = cv2.flip(frame, 1)
       cv2.imshow('example window', frame)

       if cv2.waitKey(1) & 0xFF == ord('q'):
           cleanup(cap)
```

Listing 3: This code fragment highlights the landmarks detected in the frame using draw_landmaks_on_image() [2] function

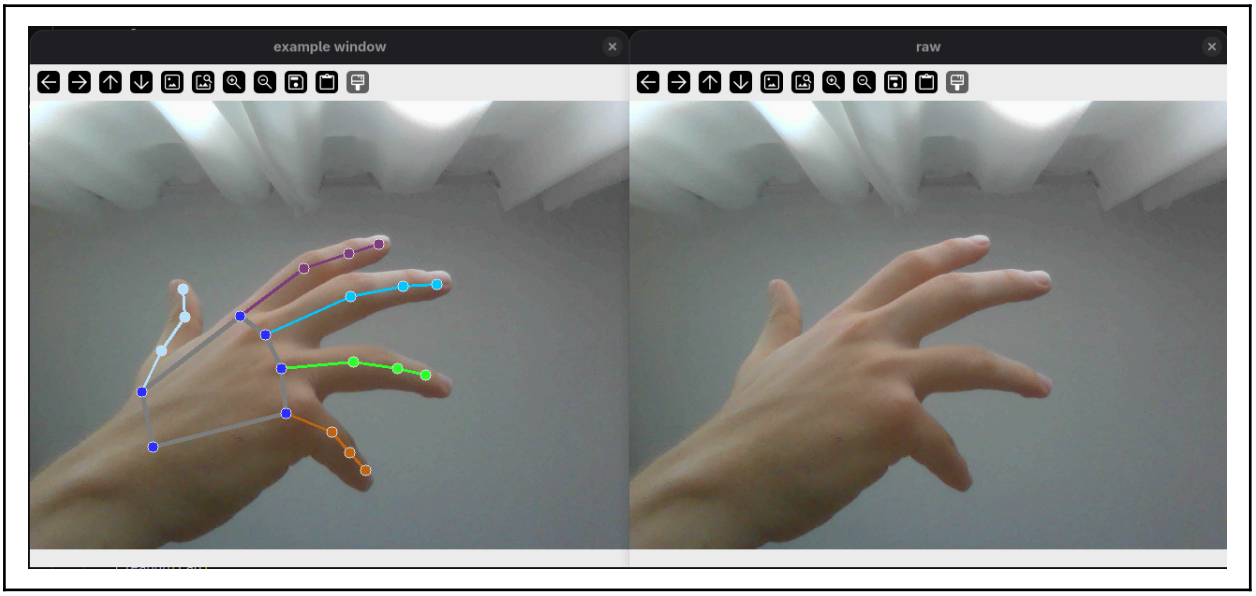


Figure 3: How landsmarks correspond to hand joints,

MediaPipe works by combining palm detection and handland marking algorithms. First a quick palm detection algorithm is run to crop fragments with hands on them and then precise landmarking algorithm is used. Palm detection is only run on the first frame or when confidence of hand being in the region is deemed too low by the hand landmarking algorithm.

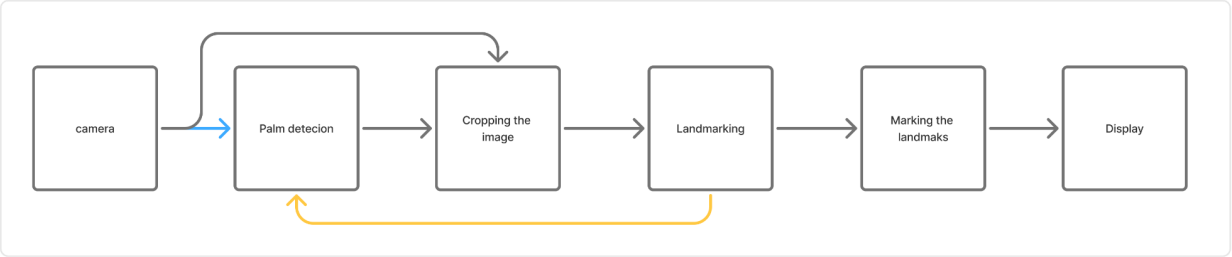


Figure 4: A simplified MediaPipe [4] pipeline in case of the earlier program. Palm detection is only run on the first frame or when the landmarking algorithm has low confidence - this is marked by yellow and blue arrows respectively

In the program we only use MediaPipe to get the landmarks and optionally display them onscreen.

MediaPipe [2] has several optional launch parameters.

num_hands=2 simply sets the maximum number of hands detected at the time it's 1 by default but we need both hands for proper detection so we set the value to 2. min_hand_detection_confidence=0.75, min_tracking_confidence=0.75 are responsible for setting minimal hand classifier and landmarker confidence respectively. The landmarking classifier forces the detection to rerun if the confidence is lower than the threshold. I experimented with several values apart from the default 0.5 and I haven't noticed the detection quality getting much better, above 0.8 I started noticing a drop in performance though. running_mode=vision.RunningMode.VIDEO is responsible for setting the running mode. Other

options are IMAGE and LIVESTREAM. The livestream and video options use information from previous frames and also don't run the hand detection every frame which is optimal for performance. I chose the video option and not stream despite technically using a livestream because livestream requires us to work with async code which is very poorly documented and video mode works good-enough in our case.

5.3 Classifying the notes with scikit-learn

Scikit learn is a library that offers an abundant set of machine learning solutions. In our program we use it for creating a note classifying model by providing it with normalized marks from MediaPipe. Specifically we train a random forest model which is a model commonly used for basic classification.

Random forest classifier [5] works by creating a set of classifying trees (100 by default) each with a random set of samples and features, then, through majority voting it chooses which group does the class fall into. This classifier for its relative simplicity and not being too heavy on compute resources.

The description of this method may look intimidating however the whole model training is just preparing an excel file and calling the training fit function.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib

df = pd.read_csv("recorder_dataset.csv")
X = df.drop('label', axis=1)
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y) # split in training adn test sets

model = RandomForestClassifier()
model.fit(X_train, y_train) # train model
y_pred = model.predict(X_test) # classify test data
accuracy = accuracy_score(y_test, y_pred)

print(f"accuracy: {accuracy * 100}%")
print(classification_report(y_test, y_pred))
joblib.dump(model, "recorder_note_model.pkl") #save the model
```

Listing 4: The figure shows model training. Not as hard as it may seem.

The accuracy depending on the data provided was always above 95% which might seem very impressive but it will be explained later what else this could mean.

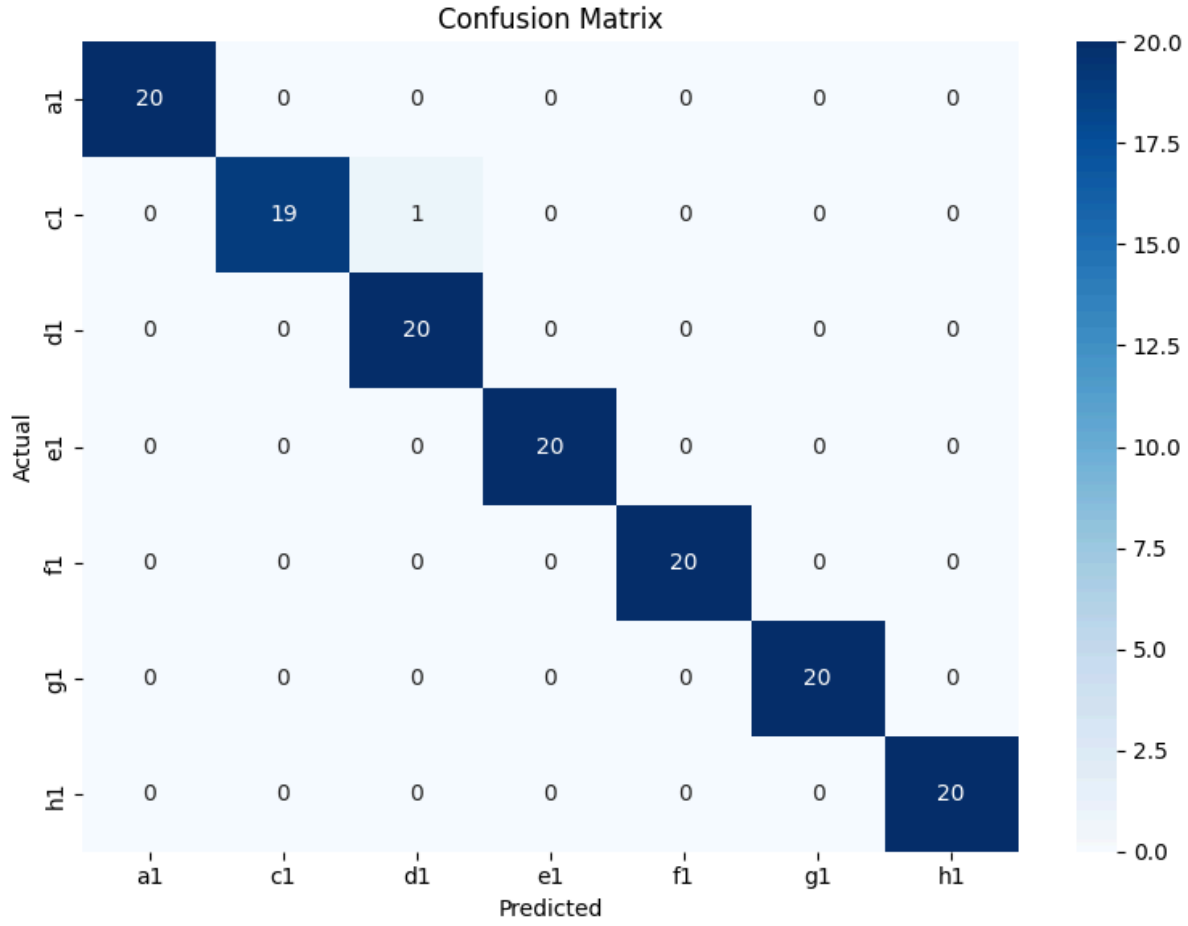


Figure 5: confusion matrix showing that the model performs well (180 samples used for training 20 for testing per note)

5.4 Gluing it together

The whole application consists of 3 separate programs:
capture.py is used to capture the data and save it to an excel file. It captures a specified number of samples per note. It only captures a note if both hands are detected on screen, it also waits for a specified number of frames before another capture. This ensures that the samples are more varied - it doesn't make sense to capture every frame since not much has changed in between just 2 frames. It also has a cooldown period between each note. If the user doesn't have enough time to change their finger position between the notes it may result in capturing incorrectly labeled data. It saves the landmark coordinates into a csv file with their corresponding labels. Note that both capture and sandbox programs use the video mediapipe option, despite capture introducing at least 20 frames of delay between captures. For some reason capturing the data with the image option and later using the video option in sandbox leads to very bad classification.

1. Get frame from the camera
2. Get landmarks
3. Normalize landmarks
4. Put them in a csv sheet
5. Repeat until captured a specified number of samples per note

Listing 5: Flow of capture.py

train_model.py is a script responsible for training the model it loads and prepares a csv file further splitting it into training and evaluation sets. After training the model on the training set it prints the accuracy score based on the evaluation set predictions.

1. Prepare csv file
2. Split the data into training and test set
3. train the model
4. evaluate the model

Listing 6; Flow of train_model.py

sandbox.py - this script allows the user to put their fingers on the flute and see which note the model thinks is being played. It also has an option to pick a random note for the user to play.

1. Get frame from the camera
2. Get landmarks
3. Normalize landmarks
4. Provide the data to the model
5. Display the output of the model formatted according to the active mode (game or sandbox)

Listing 7: Flow of sandbox.py

Each program can be closed using q, the visibility of the landmarks can be toggled using l, the reference image visibility can be toggled using r and game mode can be triggered with the g key in the sandbox script. A note displayed in lower case means that it was the last detected note, but the model cannot provide the current prediction because either one or both hands are missing.

Some important considerations:

1. Before saving the data to excel or loading it into the model for classification data is always normalized, the normalization consists of 2 steps. The first step is setting the wrist as a reference point that removes the necessity of the hand to be in the same exact spot in a reference to the camera every time. The second step is dividing everything by the distance from the wrist to the middle finger. This is crucial because not only does it allow the user to move their hands closer and further from the camera but also enables people with hands of different size to use the program without prior model training and data capture - although training is still highly recommended. Note that while MediaPipe "normalizes" the landmark

coordinates it normalizes them only relative to image size (no matter the size the image width and height are always 1)

$$P_i = \frac{P_i - P_0}{||P_9 - P_0||}$$

Figure 6: Formula for normalized version of the point's I coordinates where P_9 is the middle finger starting point and P_0 is point corresponding to wrists coordinate

2. The landmarks of the left pinky and thumbs are always ignored, they are not actively used when playing the recorder and for this reason they are omitted when saving the data, training the model and predicting the note. The wrists coordinates are also skipped because, due to the process of normalization described earlier, they would always be equal to 0.
3. When a note is being predicted a buffer of ten frames is used - this ensures that the model doesn't rapidly switch between notes because of a single misprediction. While this does improve the switching situation it may cause a bit of delay when the user actually switches their finger position.
4. The program currently only supports the first octave. This program is just a demonstration and capturing the data for more notes is very time consuming. The program may be expanded to support this functionality in the future.

MediaPipe limitations:
The whole process relies on MediaPipe for its most important functionality, this means that it has the same limitations as media pipe.

1. It doesn't work perfectly - sometimes MediaPipe is just wrong. Especially when the finger is being occulted by another finger or some different object - the flute in our case.

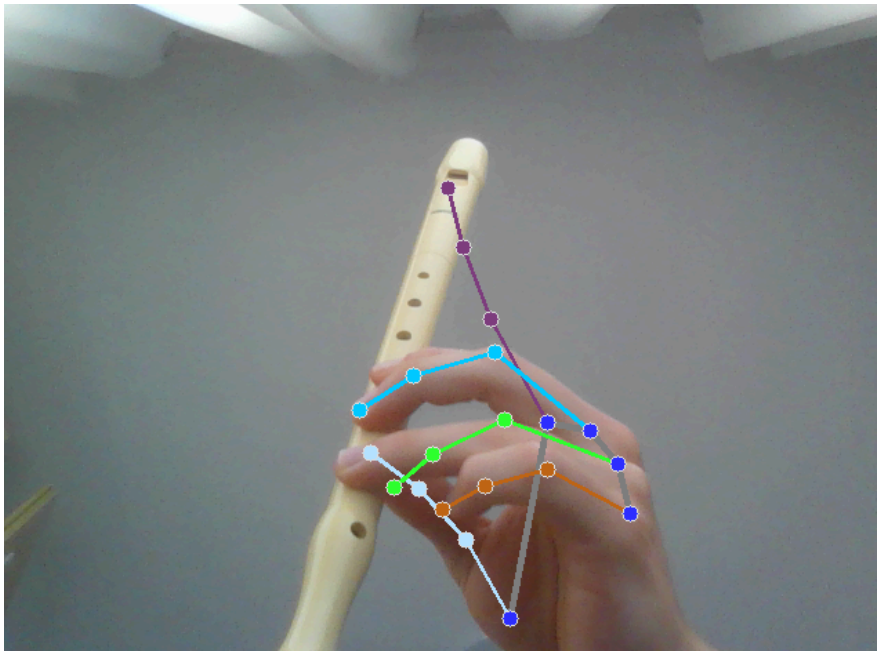


Figure 7: the flute recognized as an index finger

2. The camera quality and lighting also plays a role. MediaPipe works better with sharper images and more light.
3. when the hand is side to the camera sometimes it's not detected at all, which isn't really surprising since MediaPipe relies on one camera only. Humans don't have such problems since they have a set of 2 eyes - placing a hand like that in front of both eyes at the same time is simply impossible.

Other limitations:

1. Other notes were also not implemented because of the fact that other octaves sometimes require the right thumb hole (the one at the bottom of the flute) to be partially open - this is extremely hard to detect using just the frontal camera. While it's probably achievable with enough samples by detecting micro movements of the thumb it wouldn't probably work very well and would probably only work on the person who provided the samples. In our case it wouldn't work since we remove the thumb landmarks during the normalization process
2. The training data leaves a lot to be desired. First the number of samples per note is not that big, I usually used 200 hundred samples per note. This is due to the fact that there is no open set of samples with labels that I could use. Another problem is that I was the only person that provided the samples, there is always the same set of hands, in the same room, from a similar perspective of my desk. This is only proven further by the model's accuracy. While it may seem ~98% accuracy is something to aim for, I think it rather proves something on the contrary - that the data was too similar and not enough changes were made throughout the training. However if many changes were introduced there may be too little data for the model to not get confused between different notes. It also proves that there should be more people sampled. I have my own grip for each note and this causes the model to be overly reliant on my playing style. The only solution to these problems is having a much bigger set of data with multiple people participating. This can be easily demonstrated on a below example:

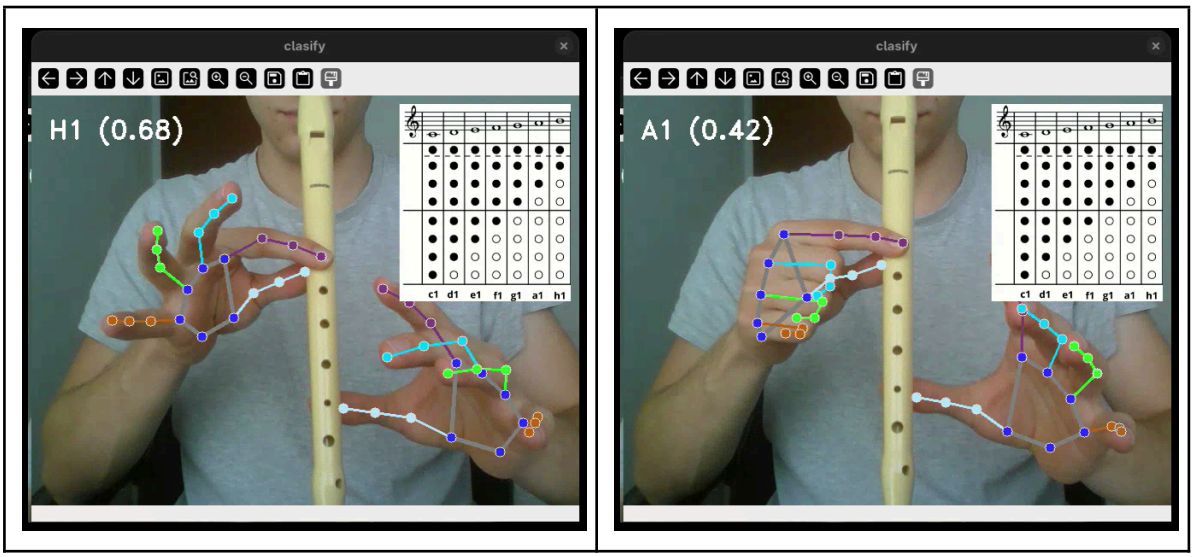


Figure 8: Note H mislabeled

We can clearly see that the user held his fingers in the same position (pointing up) when playing the H note during training. Now the system associates fingers up with the H note being played

so once the user curls his fingers the system indicates note A being played despite only the top hole being covered.

6 Possible future expansions

Obviously more functionalities could be added. We could create more interesting games which would test the users' note knowledge. Other machine learning models could be evaluated, we don't need to use random forest, what is more it's possible that some of them would even work better than random forest. Support for more notes could be added for the most part is just a matter of adding them to the note list but the issue of right thumb needs to be solved somehow if we don't want to have octaves missing notes.

Another question is the rotation support. We don't really normalize rotation wise - if the user tilts the flute a bit more than during data collection the note won't be recognized correctly. The situation is the same when the user is upside down or rotated left or right. This is not as hard to implement as the thumb positioning but it still poses quite a challenge.

Once these issues are resolved the system should be quite impressive, even more than it is already. I also encourage others to test other wind instruments with this program or maybe even string instruments - the program doesn't care what instrument is being played as long as it has the training data for it.

7 Conclusion

The article describes the basic functioning of the program, of course code analysis for full understanding but this shouldn't be a problem now. It shows how various libraries can be combined together to create an impressive multimedia application, how one does not need special hardware to recognize gestures (which was usually done by equipment like kinect) and how machine learning is made simple to use by the likes of scikit-learn.

8 Bibliography and references

All consulted April 2026

[1] MediaPipe github <https://github.com/google-ai-edge/mediapipe>
[2] Google Ai edge <https://ai.google.dev/edge/mediapipe/solutions/guide> + its collab code
[3] MediaPipe documentation <https://mediapipe.readthedocs.io/en/latest>
[4] Hand tracking paper <https://arxiv.org/pdf/2006.10214>
[5] Sci-kit learn documentation <https://scikit-learn.org/stable>
[6] Sistemas inteligentes UPV course
[7] OpenCV <https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html>